



UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Computación Paralela y Distribuida

Semestre II — 2025

Informe — Examen Parcial 1

Consultoría HPC usando OpenMP

Problema 1: El Histograma Masivo

Consultora: Ram Industries

Integrante: Diego Alejandro Ramírez Velásquez — Carné 23601

1. Contexto y Datos

1.1 El problema

Recibimos un arreglo unidimensional con **miles de millones de mediciones de temperatura** en punto flotante (°C). El encargo es clasificar cada medición en una de **100 cubetas** de igual ancho y contar cuántas mediciones caen en cada rango (un histograma). El resultado debe ser **exacto** (ninguna medición se pierde ni se cuenta dos veces) y calcularse **lo más rápido posible**.

1.2 Propuesta secuencial (algoritmo base)

El algoritmo base es directo y recorre el arreglo una sola vez — es **O(N)**:

1. Definir el rango de temperaturas esperado: $[-50\text{ }^{\circ}\text{C}, 50\text{ }^{\circ}\text{C}]$.
2. Ancho de cada cubeta = $(50 - (-50)) / 100 = 1.0\text{ }^{\circ}\text{C}$.
3. Para cada medición x : calcular $\text{idx} = (\text{int})((x - (-50)) / 1.0)$, recortarlo a $[0, 99]$ y hacer $\text{hist}[\text{idx}]++$.
4. Verificación: la suma de las 100 cubetas debe ser exactamente N .

1.3 Justificación de los datos de prueba

- **Origen:** como no disponemos de un dataset físico de miles de millones de lecturas, generamos los datos de forma **sintética y reproducible** con un PRNG rápido (xorshift64) de semilla fija. Así la versión secuencial y la paralela procesan **exactamente los mismos datos** y la comparación es justa. Con distribución uniforme cada cubeta recibe $\sim N/100$ mediciones.
- **Tamaño de la muestra:** por defecto **200,000,000** (200 M) de float (~ 0.8 GB). En una máquina de 16 GB de RAM es un punto seguro y suficientemente grande para que el trabajo domine sobre el overhead de crear hilos.
- **Estructura en memoria:** un solo arreglo contiguo float `datos[N]` (acceso secuencial, ideal para el prefetch de la cache) y un arreglo pequeño long long `hist[100]` para los conteos.

2. Estrategia de Paralelización

2.1 El reto: condición de carrera

Al repartir el bucle entre hilos, varios hilos intentan ejecutar `hist[idx]++` sobre las **mismas** 100 posiciones al mismo tiempo. `hist[idx]++` **no es atómico** (es leer $\rightarrow$ sumar $\rightarrow$ escribir), así que dos hilos pueden leer el mismo valor y perder incrementos $\rightarrow$ el conteo sale **mal**. Esto es una **race condition**.

2.2 Opciones evaluadas

Enfoque	Idea	Correcto	Velocidad
A. #pragma omp atomic	proteger cada <code>hist[idx]++</code>	Sí	Muy lento: serializa cada incremento; con solo 100 cubetas hay muchísima contención.
B. Histogramas privados	cada hilo cuenta en su copia local y al final se combinan	Sí	Rápido: cero contención en el bucle caliente.
C. <code>reduction(+:hist[:100])</code>	OpenMP hace el enfoque B automáticamente	Sí	Rápido y limpio (menos código, menos errores).

Decisión: enfoque C. Cada hilo obtiene una copia privada del histograma (solo 100 enteros, cabe holgado en la cache L1), acumula ahí sin estorbar a nadie, y al terminar OpenMP suma todas las copias. Es la forma idiomática y de mejor rendimiento para una reducción.

2.3 Directivas de OpenMP utilizadas

```
#pragma omp parallel for schedule(static) reduction(+:hist[:NUM_CUBETAS])
for (long long i = 0; i < N; i++) {
    int idx = (int)((datos[i] - TEMP_MIN) / ancho_cubeta);
    if (idx < 0) idx = 0; else if (idx >= NUM_CUBETAS) idx = NUM_CUBETAS - 1;
    hist[idx]++;
}
```

- **`parallel for`**: reparte las N iteraciones entre los hilos.
- **`reduction(+:hist[:NUM\_CUBETAS])`**: reducción **de arreglo** (OpenMP 4.5+). Elimina la condición de carrera sin `atomic` ni `critical`.
- **`schedule(static)`**: como cada iteración cuesta lo mismo, no hay desbalance de carga; el reparto estático en bloques contiguos es el más barato y respeta la localidad de cache.

2.4 Cómo evitamos race conditions y desbalance

- **Race conditions**: resueltas por la reducción (copias privadas + combinación gestionada por OpenMP).
- **Desbalance de carga**: inexistente por diseño (trabajo uniforme) → `schedule(static)` es óptimo.

3. Resultados y Métricas

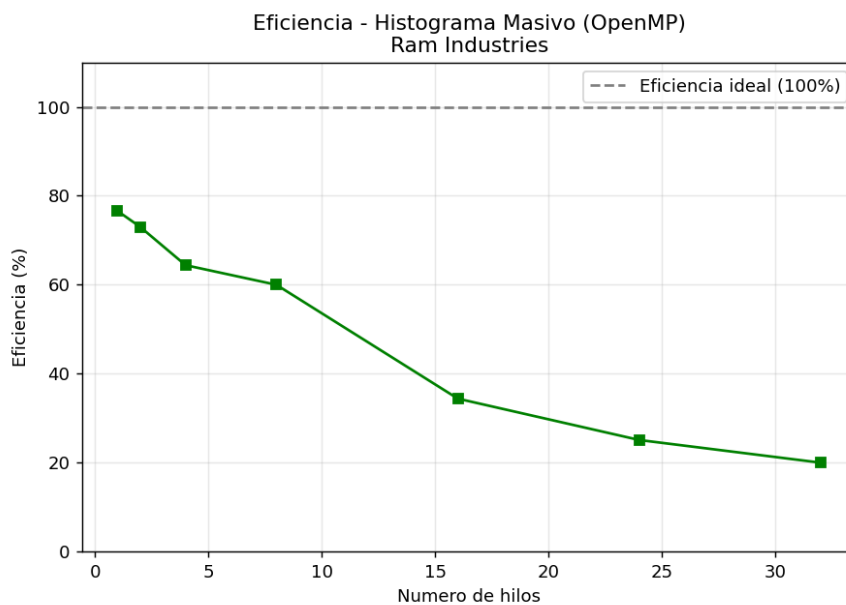
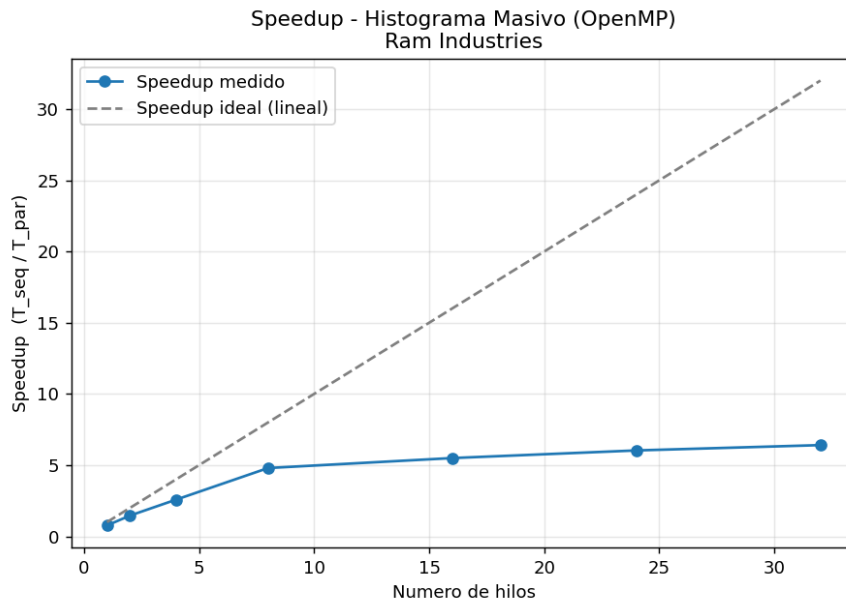
3.1 Entorno de pruebas

- **Equipo**: ASUS ROG G16
- **CPU**: Intel Core **i9-14900HX** (8 núcleos P + 16 núcleos E = 24 núcleos, 32 hilos)
- **RAM**: 16 GB · **Compilador**: GCC con `-O2 -fopenmp` · **N**: 200,000,000 mediciones

3.2 Métricas

Speedup = $T_{\text{secuencial}} / T_{\text{paralelo}}(p)$ · **Eficiencia** = $\text{Speedup}(p) / p$. Tiempo base secuencial: **$T_{\text{secuencial}} \approx 0.1639$ s** (con 1 hilo la versión OpenMP es más lenta que la secuencial pura por el overhead de crear el equipo de hilos).

Hilos (p)	Tiempo (s)	Speedup	Eficiencia
1	0.2140	0.77×	76.6%
2	0.1123	1.46×	73.0%
4	0.0637	2.58×	64.4%
8	0.0342	4.80×	60.0%
16	0.0298	5.50×	34.4%
24	0.0272	6.03×	25.1%
32	0.0256	6.41×	20.0%



3.3 Análisis de los resultados

- **El speedup crece rápido hasta 8 hilos y luego se aplana.** De 1→8 hilos sube casi linealmente (hasta **4.80×**), pero de 8→32 apenas pasa de **4.80×** a **6.41×** a pesar de cuadruplicar los hilos. Razones: (1) los **8 P-cores** hacen el grueso del trabajo y los **E-cores** (más lentos) aportan menos; (2) el kernel es **memory-bound** — por dato solo hace una división y un ++, así que el cuello de botella pasa a ser el **ancho de banda de memoria** al recorrer ~0.8 GB.
- **La eficiencia se desploma** de ~60% con 8 hilos a **20% con 32 hilos**: agregar hilos deja de valer la pena. Este es el argumento de consultoría — "**más hilos**" **no es igual a "más rápido"**: el punto óptimo está alrededor de **8 hilos**, y forzar 32 solo aporta ~34% más de velocidad a costa de triplicar el uso de núcleos.
- **Correctitud**: en toda corrida se verifica $\text{suma de cubetas} = N$ (imprime OK), demostrando que la reducción eliminó la condición de carrera.

4. Conclusiones

- El histograma masivo se paraleliza de forma natural con una **reducción de arreglo** en OpenMP, que elimina la condición de carrera sin sacrificar velocidad.

- `schedule(static)` es la elección correcta porque la carga es uniforme.
- La ganancia real está **acotada por el ancho de banda de memoria** y por la diferencia P-core/E-core, no por el número de hilos: el valor de la consultoría es saber **cuándo dejar de agregar hilos** (aquí, ~8).